USFQ Ecuador

Organización de Computadoras · Proyecto Final

Buscaminas en MARIE.js: Implementación de un Juego Clásico sobre Arquitectura Educativa con Generador Externo en Python

Mateo Díaz · Ayelen Jaramillo

Universidad San Francisco de Quito · Ingeniería en Computación

MARIE.

256 c

github.com/

Resumen — Se implementó el juego Buscaminas sobre la arquitectura educativa MARIE.js mediante una solución híbrida de dos capas. Un generador externo en **Python** construye el tablero 16×16, coloca minas aleatoriame conteos de vecinos y exporta los datos en formato DEC directamente incrustados en la plantilla `.mas`. El programa en **MARIE** gestiona la interacción: lee entradas del jugador, revela celdas, maneja banderas y renderiza el est paleta de colores del display 16×16. Un sistema de **máscaras binarias** permite cambiar la geometría del tablero (llena, rombo, cruz, marco, personalizada) sin modificar la lógica del juego.

INTRODUCCIÓN

El Buscaminas es un juego combinatorio en el que el jugador debe descubrir celdas de un tablero evitando las que contienen minas. Cada celda segura muestra la cantidad de minas en su vecindad, permitiendo inferir la posición del peligro.

Implementarlo en **MARIE.js** presenta restricciones significativas: arquitectura acumulador único, sin recursión, sin generación aleatoria nativa y memoria lineal de acceso directo.

Sin recursión · Sin RNG nativo · Memoria lineal · Entrada secuencial

La solución adopta una **arquitectura de dos capas**: Python resuelve la complejidad generativa y MARIE ejecuta la lógica interactiva.

METODOLOGÍA · GENERADOR PYTHON

El generador sigue un pipeline modular de tres etapas que produce dos estructuras: **TABLERO_REAL** y **SHAPE_PRESET**.

```
func generar_tablero(filas, cols, minas, semilla):
  tablero = crear_tablero(filas, cols) / zeros
  tablero = colocar_minas(tablero, minas, semilla)
  tablero = calcular_vecinos(tablero)
  retornar tablero

func calcular_vecinos(tablero):
  para cada celda (f, c) no mina
    conteo = 0
    para (nf,nc) en vecinos_de(f, c):
      si tablero[nf][nc] == -1  conteo ++
    tablero[f][c] = conteo
```

Convención de valores: `-1` = mina, `0..8` = conteo de vecinos.

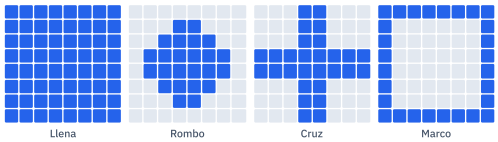
```
index = fila × 16 + columna   (0 ≤ index < 256)
```

La **exportación automática** reemplaza los marcadores __TABLERO_REAL__ y __SHAPE_PRESET__ en la plantilla base, generando el archivo `.mas` listo para MARIE.js sin intervención manual.

256	20+	5
celdas totales	minas típicas	formas

METODOLOGÍA · MÁSCARAS DE FORMA

Una **máscara binaria** de 16×16 determina qué celdas pertenecen al área jugable. Esto desacopla completamente la geometría del tablero de la lógica del juego en MARIE.




```
func generar_mascara(tipo, filas, cols):
  si tipo == "rombo"
    radio = min(filas, cols) / 2
    activa si |i-cf| + |j-cc| ≤ radio
  si tipo == "cruz"
    activa si |i-cf| ≤ grosor o |j-cc| ≤ grosor
  si tipo == "marco"
    activa si i < g o i ≥ f-g o j < g o j ≥ c-g
```

METODOLOGÍA · LÓGICA MARIE.JS

El programa en MARIE implementa el ciclo completo del juego. Cada turno consume **tres entradas consecutivas**:

```
fila (0-15):   6
col (0-15):    10
acción:        1 → revelar celda
```

```
fila (0-15):   3
col (0-15):    7
acción:        2 → colocar / quitar bandera
```

Mapa de memoria en tiempo de ejecución:

0x000	Código + Vars	Programa y variables globales
REAL	TABLERO_REAL	256 valores −1..8 (Python)
PRESET	SHAPE_PRESET	256 valores 0/1 (Python)
0x600	TABLERO_VIS	Estado visible (runtime)
0xF00	DISPLAY 16×16	Buffer de color (render)

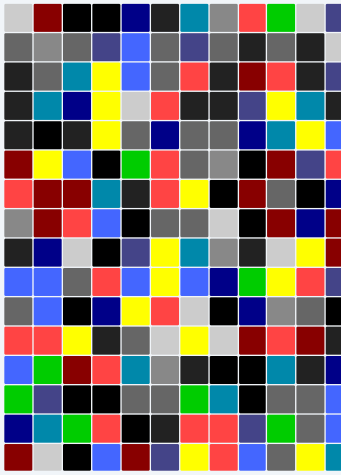
Las rutinas clave son APPLY_SHAPE_PRESET (copia máscara al inicio) y CALC_SAFE_TARGET (cuenta celdas válidas sin mina para detectar victoria).

```
REVEALED_COUNT == SHAPE_SAFE_TARGET  →  WIN
```

RESULTADOS

El sistema genera tableros válidos de Buscaminas y los en MARIE.js. El display 16×16 representa el estado por

Vista del tablero 16×16 en el display de MAR



- Celda oculta
- Mina revelada
- 1 vecino
- 3 vecinos
- Fuera de forma
- Bandera
- 0 vecinos
- 2 vecinos
- 4–8 vecinos
- Celda activa

Salida por consola al revelar una celda segura: el valor colocar bandera: `FLAGS: XX`; fin del juego: `WIN` o `LOSE` + banderas.

3	WIN
entradas por turno	/ LOSE detectados

CONCLUSIONES

- **Solución híbrida exitosa.** La separación Python/MAR arquitecturas simples pueden ejecutar aplicaciones i cuando se delega la lógica generativa a herramientas
- **Desacoplamiento forma/lógica.** El sistema de máscar geometría del tablero sin tocar el código MARIE, evider modular dentro de un entorno limitado.
- **Limitaciones de MARIE.** La ausencia de recursión impi fill automático. La falta de estructuras dinámicas obliga con direcciones explícitas, incrementando la complejid estado.
- **Reproducibilidad.** El notebook autosuficiente permit con distintos parámetros (minas, forma, semilla) desc dependencias locales.

REFERENCIAS

1. M. Null y L. Lobur, *The Essentials of Computer Organization* Bartlett Learning.
2. MARIE.js Simulator. Disponible en: `https://marie.js.org`
3. Python Software Foundation, *Python 3 Documentation*. Disp `https://docs.python.org`